

DEPARTMENT OF COMPUTER SCIENCE



University of
Reading

CS1IP: Imperative Programming

Week 11: More collections and Streams

Learning Outcomes

- Which are other features of `ArrayList` and `LinkedList` ?
- What is a Java Lambdas?
- What is a Java Streams?

Recap on `ArrayList` and `LinkedList`

Recap on `ArrayList` and `LinkedList`

Creating ArrayLists in Java

- Using the diamond `<>` operator (with type at the right)

```
ArrayList<String> list1 = new ArrayList<String>();  
LinkedList<String> list2 = new LinkedList<String>();
```

- Using `var`

```
var list1 = new ArrayList<String>();  
var list2 = new LinkedList<String>();
```

- Using the diamond `<>` operator (without type at the right)

```
var list1 = new ArrayList<String>;  
var list2 = new LinkedList<String>;
```

Recap on ArrayList and LinkedList

add()

- ArrayList:

```
var list = new ArrayList<String>;  
list.add("Apple");
```

- LinkedList:

```
var list = new LinkedList<String>  
list.add("Banana");
```

Recap on ArrayList and LinkedList

Populating a collection when creating

- ArrayList:

```
var list1 = new ArrayList<String>(Arrays.asList("Alice", "Bob", "Charlie"));
```

- LinkedList:

```
var list2 = new LinkedList<String>(Arrays.asList("Alice", "Bob", "Charlie"));
```

Recap on `ArrayList` and `LinkedList`

Sorting a collection

- `ArrayList`:

```
var list1 = new ArrayList<String>(Arrays.asList("Bob", "Charlie", "Alice"));  
Collections.sort(list1); // "Alice", "Bob", "Charlie"
```

- `LinkedList`:

```
var list2 = new LinkedList<String>(Arrays.asList("Bob", "Charlie", "Alice"));  
Collections.sort(list2); // "Alice", "Bob", "Charlie"
```

Recap on `ArrayList` and `LinkedList`

Accessing Elements by Index: `get(int index)`

- `ArrayList`:

```
var list1 = new ArrayList<String>(Arrays.asList("Alice", "Bob", "Charlie"));
String name = list1.get(2);
```

- `LinkedList`:

```
var list2 = new LinkedList<String>(Arrays.asList("Alice", "Bob", "Charlie"));
String name = list2.get(2); // Slower for large lists
```

i Important: `get` has a fast access with `ArrayList` is ideal for scenarios like reading configuration lists.

Recap on `ArrayList` and `LinkedList`

Removing Elements `remove(int index)`

- `ArrayList`:

```
var list1 = new ArrayList<String>(Arrays.asList("Alice", "Bob", "Charlie"));  
list.remove(1); // Shifts elements
```

- `LinkedList`:

```
var list2 = new LinkedList<String>(Arrays.asList("Alice", "Bob", "Charlie"));  
list.remove(1); // More efficient for large lists
```

i Important: Frequent deletions in `ArrayList` can degrade performance due to shifting elements.

Performance Considerations

Function	ArrayList	LinkedList
Add at End	Fast	Fast
Add in Middle	Slow (shifting required)	Fast
Get by Index	Fast (random access)	Slow (traversal required)
Remove in Middle	Slow (shifting required)	Fast

Summary:

- Use ArrayList for random access and frequent reads.
- Use LinkedList for frequent insertions and deletions.

More about `ArrayList` and `LinkedList`

Inserting at Specific Index

add(int index, E element)

- ArrayList:

```
var list1 = new ArrayList<String>(Arrays.asList("Alice", "Bob", "Charlie"));  
list1.add(1, "Alan"); // Slower for large lists
```

- LinkedList:

```
var list2 = new LinkedList<String>(Arrays.asList("Alice", "Bob", "Charlie"));  
list2.add(1, "Alan");
```

i Important: For large arrays, inserting into the middle of an `ArrayList` can be inefficient due to shifting elements.

Checking for Existence

`contains(E element)`

- ArrayList:

```
var list1 = new ArrayList<String>(Arrays.asList("Alice", "Bob", "Charlie"));  
list1.contains("Bob"); // true  
list1.contains("Alan"); // false
```

- LinkedList:

```
var list2 = new LinkedList<String>(Arrays.asList("Alice", "Bob", "Charlie"));  
list2.contains("Bob"); // true  
list2.contains("Alan"); // false
```

i Use Case: Checking for membership in a small collection where speed is not critical

Searching for Elements

indexOf()

- ArrayList:

```
var list1 = new ArrayList<String>(Arrays.asList("Alice", "Bob", "Charlie"));  
int index = list1.indexOf("Alice"); // 0
```

- LinkedList:

```
var list2 = new LinkedList<String>(Arrays.asList("Alice", "Bob", "Charlie"));  
int index = list2.indexOf("Charlie"); // 2
```

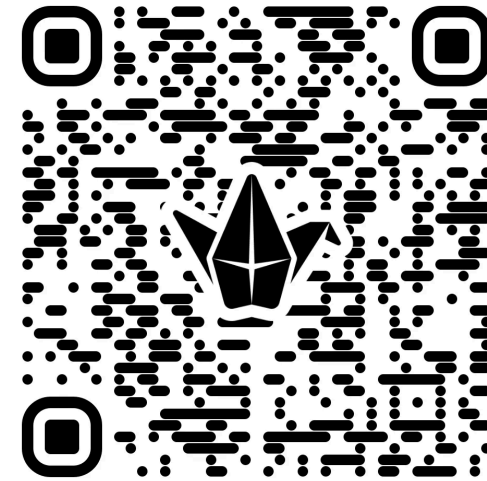
i Use Case: Finding the index of an element when duplicates are not expected.

Quiz 1 ([click here](#) or scan QRCode)

What is the output of the following code?

```
var list = new ArrayList<String>();  
list.add("Apple");  
list.add("Banana");  
list.add(1, "Cherry");  
list.add("Date");  
System.out.println(list.get(0));  
System.out.println(list.get(2));  
System.out.println(list.get(3));
```

- a. Apple, Banana, Date
- b. Apple, Cherry, Date
- c. Cherry, Banana, Apple
- d. Banana, Cherry, Apple



Explanation

1. `list.add("Apple");` → `["Apple"]`
2. `list.add("Banana");` → `["Apple", "Banana"]`
3. `list.add(1, "Cherry");` → `["Apple", "Cherry", "Banana"]`
4. `list.add("Date");` → `["Apple", "Cherry", "Banana", "Date"]`

When calling `get` :

- `list.get(0)` returns `"Apple"` .
- `list.get(2)` returns `"Cherry"` .
- `list.get(3)` returns `"Date"` .

Iterating over elements

Using `forEach` SYNTAX

```
list.forEach(METHOD_REFERENCE)
```

What are method references?

- It provide a shorthand syntax to refer to methods.
- The General Syntax is `ClassName::methodName`
- Example is `System.out::println`

Iterating over elements

Using `forEach`

- ArrayList:

```
var list1 = new ArrayList<String>(Arrays.asList("Alice", "Bob", "Charlie"));  
list1.forEach(System.out::println);
```

- LinkedList:

```
var list2 = new LinkedList<String>(Arrays.asList("Alice", "Bob", "Charlie"));  
list2.forEach(System.out::println);
```

In many causes we need a quick method to used

❓ Is there quickly create method but without do its entire definition?

Lambdas

Lambdas in Java

What is a Lambda?

- A lambda expression is a concise way to represent an anonymous function.
 - **anonymous function = function without name**
- Allows you to pass behavior as a method argument.
- introduced in Java 8.

Take-Home message

- lambda allow functions as **First-Class Citizens**
- You can treat functions like values and pass them around.

Lambda Syntax

Without body

```
(parameters) -> body
```

Example

```
x -> System.out.println(x)
```

i Without the body it is like void function

Comparing a Lambda (without body) and a traditional function

lambda to print a variable:

```
x -> System.out.println(x)
```

equivalent function to print a variable:

```
void print(String x){  
    System.out.println(x)  
}
```

i You can see that lambda is more concise

Lambda syntax

With body (return block)

```
(parameters) -> { body }
```

```
x -> { return x * 2};
```

another example

```
(a, b) -> {  
    int sum = a + b;  
    return sum;  
};
```

Comparing a Lambda (with body) and a traditional function

lambda to print a variable:

```
x -> { return x * 2};
```

equivalent function to print a variable:

```
int multi2(int x){  
    return x * 2;  
}
```

i You can see that lambda is more concise

ArrayList.forEach with lambda

using function reference

```
var list1 = new ArrayList<String>(Arrays.asList("Alice", "Bob", "Charlie"));  
list1.forEach(System.out::println);
```

now lambda

```
var list1 = new ArrayList<String>(Arrays.asList("Alice", "Bob", "Charlie"));  
list1.forEach(name -> System.out.println(name));
```

Adding elements to another collection

```
var names = new ArrayList<String>(Arrays.asList("Alice", "Bob", "Charlie"));  
var upperCaseNames = new ArrayList<String>();  
names.forEach(name -> upperCaseNames.add(name.toUpperCase()));  
upperCaseNames.forEach(System.out::println);
```

Output:

```
ALICE  
BOB  
CHARLIE
```

Performing calculations and storing the result

Example: Calculating the sum of elements

```
var numbers = new ArrayList<Integer>(Arrays.asList(1, 2, 3, 4, 5));  
int sum = 0;  
  
numbers.forEach(n -> sum += n);  
  
System.out.println("Sum: " + sum);
```

Output:

```
Sum: 15
```

Example of lambda calling `Integer.parseInt`

```
var numbers = new ArrayList<String>(Arrays.asList("10", "20", "30"));  
numbers.forEach(s -> System.out.println(Integer.parseInt(s) * 2));
```

Output:

```
20  
40  
60
```

Example of lambda calling `Double.parseDouble`

```
var numbers = new ArrayList<String>(Arrays.asList("1.5", "2.75", "3.25"));  
numbers.forEach(n -> System.out.println(Double.parseDouble(n) + 1.0));
```

Output:

```
2.5  
3.75  
4.25
```

Example of lambda calling `Math.sqrt`

```
var numbers = new ArrayList<Integer>(Arrays.asList(4, 9, 16, 25));  
numbers.forEach(n -> System.out.println(Math.sqrt(n)));
```

Output:

```
2.0  
3.0  
4.0  
5.0
```

Example of lambda calling `Math.abs`

```
var numbers = new ArrayList<Integer>(Arrays.asList(-1, -2, 3, -4));  
numbers.forEach(n -> System.out.println(Math.abs(n)));
```

Output:

```
1  
2  
3  
4
```

Print modified elements of a list

Example: Doubling each element in the list

```
var numbers = new ArrayList<Integer>(Arrays.asList(1, 2, 3, 4, 5));  
numbers.replaceAll(n -> n * 2);  
numbers.forEach(n -> System.out.println(n));
```

Output:

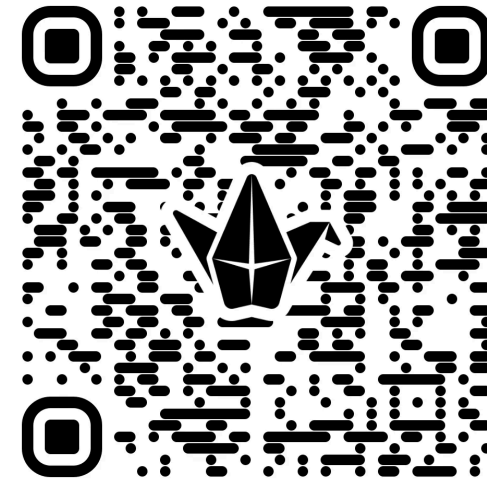
```
2  
4  
6  
8  
10
```

Quiz 2 ([click here](#) or scan QRCode)

What is the output of the following code?

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4);  
numbers.replaceAll(n -> n + 1);  
numbers.forEach(n -> n = n * 2);  
numbers.forEach(System.out::println);
```

- 2, 4, 6, 8
- 2, 3, 4, 5
- 4, 6, 8, 10
- 1, 2, 3, 4



Explanation

- **Initial List:** `numbers = [1, 2, 3, 4]`
- `replaceAll(n -> n + 1)`
 - Increases each element by 1: `[2, 3, 4, 5]`
- `forEach(n -> n = n * 2)`
 - this assignment does not modify the elements in the list because `forEach` does not support element modification
- `forEach(System.out::println)`
 - Prints the elements as they are: `2, 3, 4, 5`

Why can't `forEach` return a value directly?

The `ArrayList.forEach` is a **void method** (Terminal Operation), meaning it does not return a value.

- So you cannot use `forEach` to collect or return values directly.

However, you can achieve similar functionality by performing operations inside the lambda that indirectly "return" values, such as:

- The `forEach` method is designed like printing or manipulate existing data. But not update it!

Is there a way to edit and update the collection using lambda?

- If you need to **transform** elements and collect the results, use a `Stream`

What is Streams?

It is a sequence of elements supporting sequential and parallel operations.

- It usually use lambdas do perform operations
- It use internation iteration

Simple ways to create Streams are from the functions:

- `IntStream.range(start, end) :`

```
IntStream.range(1,5) // [1, 2, 3, 4]
```

- `IntStream.rangeClose(start, endInclusive) :`

```
IntStream.rangeClose(1,5) // [1, 2, 3, 4, 5]
```

Internal vs. External Iteration

- **External Iteration** (traditional `for` loops): When you get an iterator (`i`) and step over a list

```
for (int i = 1; i < 5; i++) {  
    System.out.println(i);  
}
```

- **Internal Iteration** (with streams): When you pass a function to a method to run over a list

```
IntStream.range(1, 5).forEach(System.out::println);
```

Example: Summing with Streams

Traditional Loop:

```
int total = 0;
for (int i = 1; i <= 10; i++) {
    total += i;
}
```

With Streams:

```
int sum = IntStream.rangeClosed(1, 10).sum();
```

How to create creating Streams in Java

The first part to create a Stream is its **Source**, which can be

- from a collection
 - using the `stream()` function from a collection
- from an array
 - using the `Arrays.stream()` function with the array as param

Creating Streams from Collections

Every Collection has `stream()` function

- Example from `ArrayList` :

```
var names = new ArrayList(Arrays.asList("Alice", "Bob", "Charlie"));
Stream<String> stream = names.stream();
stream.forEach(System.out::println);
```

- Example from `Set` :

```
var numbers = new HashSet<>(Arrays.asList(1, 2, 3));
numbers.stream().forEach(System.out::println);
```

Using `IntStream`

- Range of integers:

```
IntStream.range(1, 5).forEach(System.out::println);  
// Output: 1, 2, 3, 4
```

- From an array of primitives:

```
int[] nums = {1, 2, 3};  
IntStream.of(nums).forEach(System.out::println);
```

Creating Streams from Arrays

For arrays you need use `Arrays.stream()` function

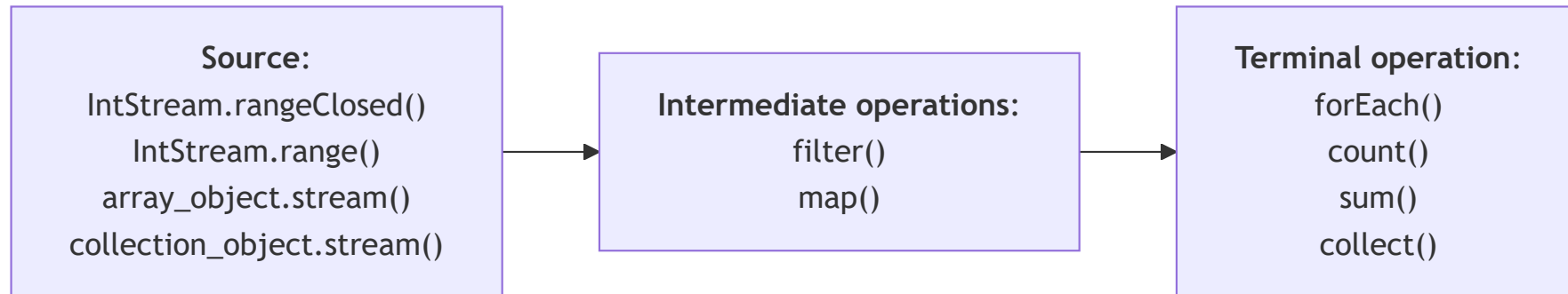
```
String[] fruits = {"Apple", "Banana", "Cherry"};  
Arrays.stream(fruits).forEach(System.out::println);
```

Streams pipelines

What is Streams pipelines?

Composed of:

- **Source** (e.g., collections, arrays)
- **Intermediate operations**
- **Terminal operations**



Intermediate Stream operations

- Transform elements in the stream.
- Examples:
 - `map()` : Apply a function to each element.
 - `filter()` : Select elements that match a condition.
 - `sorted()` : Sort elements.

`map()` apply a function to each element

Example Multiplication with Streams

Traditional loop:

```
int total = 0;
for (int i = 1; i <= 5; i++) {
    System.out.print(i*2 + " ");
}
```

With Streams:

```
IntStream.rangeClosed(1, 5).map(n -> n * 2).forEach(System.out::println);
// Output: 2, 4, 6, 8, 10
```


 Is there way to use map for a basic array?

Example `map` with Basic Array

Doubling Elements

```
int[] numbers = {1, 2, 3, 4, 5};  
int[] doubled = Arrays.stream(numbers).map(n -> n * 2).toArray();  
System.out.println(Arrays.toString(doubled));  
// Output: [2, 4, 6, 8, 10]
```

Example `map` with Basic Array

Squaring Elements

```
int[] numbers = {1, 2, 3, 4, 5};  
int[] squares = Arrays.stream(numbers).map(x -> x * x).toArray();  
System.out.println(Arrays.toString(squares));  
// Output: [1, 4, 9, 16, 25]
```

filter with Basic Array

Filtering Elements That Start with "A"

```
String[] names = {"Alice", "Bob", "Charlie"};  
String[] filteredNames = Arrays.stream(names).filter(name -> name.startsWith("A")).toArray(String[]::new);  
System.out.println(Arrays.toString(filteredNames));  
// Output: [Alice]
```

`filter()` select elements that match a condition

Traditional loop:

```
String[] names = {"Alice", "Bob", "Charlie"};
for (String name : names) {
    if (name.startsWith("A")) {
        System.out.println(name);
    }
}
// Output: Alice
```

With Streams:

```
String[] names = {"Alice", "Bob", "Charlie"};
names.stream().filter(name -> name.startsWith("A")).forEach(System.out::println);
// Output: Alice
```

`sorted()` sorts elements

Traditional Loop:

```
int[] numbers = new int[]{5, 1, 4, 3, 2}
Arrays.sort(numbers);
for (int number : numbers) {
    System.out.print(number + " ");
}
// Output: 1 2 3 4 5
```

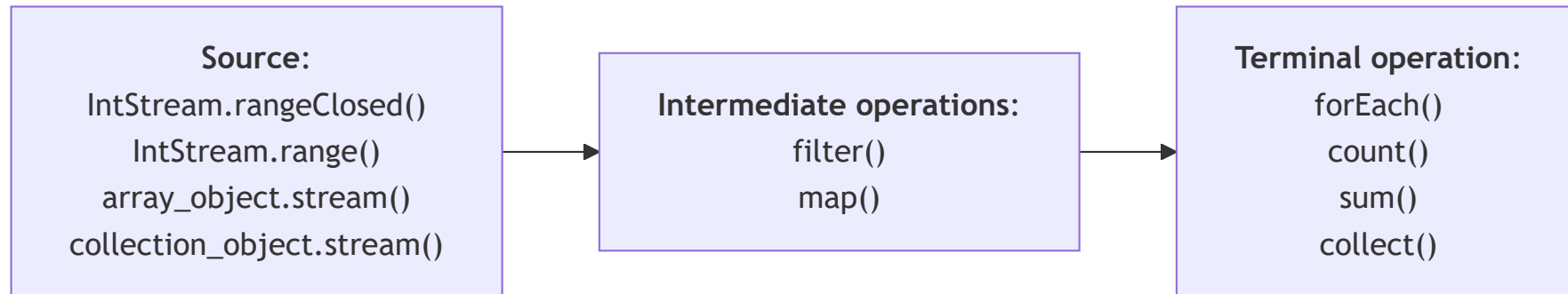
With Streams

```
int[] numbers = new int[]{5, 1, 4, 3, 2}
Arrays.stream(numbers).sorted().forEach(n -> System.out.print(n + " "));
// Output: 1 2 3 4 5
```

What is Streams pipelines?

Composed of:

- **Source** (e.g., collections, arrays)
- **Intermediate operations**
- **Terminal operations**



Terminal operations

- Produce a result or side effect.
- Examples:
 - `forEach` : Iterate through elements.
 - `count` : Count the elements.
 - `sum` : Sum the elements.
 - `collect` : Gather elements into a new collection.

Example `collect` after `map`

```
var numbers = Arrays.asList(1, 2, 3, 4, 5);  
var doubled = numbers.stream().map(n -> n * 2).collect(Collectors.toList());  
System.out.println(doubled);  
// Output: [2, 4, 6, 8, 10]
```

Example `collect` after `map`

Example using `map` :

```
var numbers = Arrays.asList(1, 2, 3, 4, 5);  
ArrayList<Integer> squares = numbers.stream().map(x -> x * x).collect(Collectors.toList());  
System.out.println(squares);
```

collect after filter

Filter elements based on a condition:

```
var names = Arrays.asList("Alice", "Bob", "Charlie");  
var filteredNames = names.stream().filter(name -> name.startsWith("A")).collect(Collectors.toList());  
System.out.println(filteredNames);
```

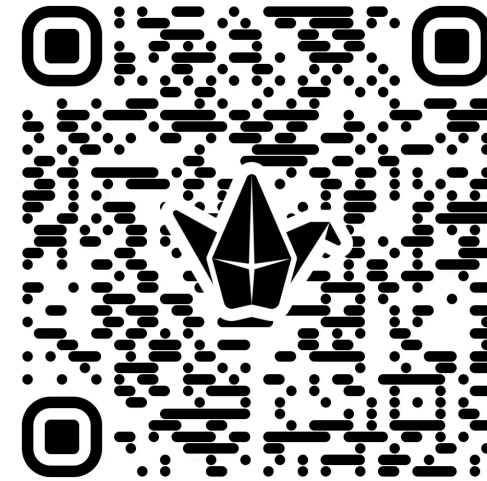
Quiz 3 ([click here](#) or scan QRCode)

What is the output of the following code?

```
var names = new ArrayList<String>(Arrays.asList("Alice", "Bob", "Charlie"));
names.stream().filter(name -> !name.toLowerCase().contains("a")).map(String::toUpperCase).forEach(System.out::println);
```

Options

- ALICE, BOB, CHARLIE, DAVID
- ALICE, CHARLIE
- BOB
- ALICE, BOB, DAVID



Explanation

- **Initial List:** `["Alice", "Bob", "Charlie", "David"]`
- `filter(name -> !name.toLowerCase().contains("e"))`
 - Excludes names containing the letter "a" in any case: `["Alice", "Charlie", "David"]`.
- `map(String::toUpperCase)`
 - Converts the remaining names to uppercase: `["BOB"]`.
- `foreach(System.out::println)` prints the filtered and uppercase names:

BOB

Benefits of Streams

- Streams **simplify processing collections**
- Use Lambdas allow the program be concise and readable.
- Easier to parallelize operations (internally done).
- Avoids common iteration errors (e.g., off-by-one).

Review Learning Outcomes

Review Learning Outcomes

Which are other features of `ArrayList` and `LinkedList` ?

- `add` , `remove` , `indexOf` , `contains`
- `forEach` is used for performing operations (e.g., modifying data, printing).

What is a Java Lambdas?

- Lambdas is a useful anonymous function syntax.

What is a Java Streams?

- Streams process sequences of elements efficiently.
- Combining lambdas and streams reduces boilerplate code.
- For transformations that return new collections, **Streams** with `map` and `collect` are more appropriate.

DEPARTMENT OF COMPUTER SCIENCE



Thank you! 😊

Do you have any questions 🙋 ?

The next lecture will cover: Security and others